CA2: Lecture Management System

Due Date 29th November

You are required to design and implement a **C# Lecture Management System**. The system will manage lecture subjects offered by a college.

You must design and implement an appropriate **class hierarchy** to represent different types of lecture subjects. Your solution must demonstrate your understanding of object-oriented programming and the C# functionality.

The system should allow users to add, view, list, edit and delete lecture subjects. The information must be saved to a file so that it is available when the program is run again.

Class Design and Inheritance

Design and implement a suitable class hierarchy for lecture subjects.

For example, you could create a base class called `Lecture` and derive specialised classes such as:

- `Programming`, `Database`, `Networking`

However, you are free to choose your own class hierarchy, provided it is appropriate to the system.

Your solution must demonstrate:

- Inheritance between classes.
- Appropriate use of constructors.
- Method overriding, using `virtual` and `override` where appropriate.
- Appropriate use of fields, properties and methods.
- Appropriate use of `static` members.

You should override at least one method in a derived class to demonstrate how different types of lecture subjects can provide their own behaviour.

Managing Lecture Subjects

The system must allow users to perform the following operations:

- **Add:** Create a new lecture subject.
- **View:** Display the details of a selected lecture subject.
- **List:** Display all lecture subjects stored in the system.
- **Edit:** Modify the details of an existing lecture subject.
- **Delete:** Remove a lecture subject from the system.

Each lecture subject should have appropriate information, such as a subject code, subject name, lecturer name and number of lecture hours. You may include additional information relevant to your design.

You should use an appropriate array or collection to manage the lecture subject objects.

Static Members

Your solution must demonstrate the use of at least one `static` field, property or method.

For example, you could use a static member to maintain a count of the lecture subjects created or to provide a shared value or operation for the system.

File Handling and Permanent Storage

Your system must save lecture subject information to a file so that the data is not lost when the application closes.

When the application starts, it should attempt to load previously saved information.

The system must support:

- Saving lecture subject data to a file.
- Loading saved data when the application starts.
- Retaining changes made when subjects are added, edited or deleted.
- Handling situations where the data file does not yet exist.

You may choose an appropriate file format and file-handling approach, subject to the techniques covered in the module.

User Interface

You must provide a user-friendly interface through which users can interact with the system.

The interface may be **console-based or graphical**, depending on the requirements specified for your class.

The interface should provide a clear menu or equivalent controls for the required operations: add, view, list, edit and delete.

The system should display useful information and feedback to the user, including confirmation messages and appropriate error messages.

Error Handling and Validation

Your system should validate user input and handle common errors appropriately.

For example:

- Preventing duplicate subject codes.
- Handling invalid menu selections.
- Ensuring that required fields are entered.
- Validating numeric values e.g. lecture hours.
- Handling attempts to view, edit or delete a subject that does not exist.
- Handling missing or invalid data files.